

# Ejercicios Propuestos #1 ¶

## Parte 1: Introducción

### Ejercicio 1

Empecemos con algo simple. Determina el resultado de las siguientes operaciones a partir de las siguientes variables. Además, identifica el tipo de dato de las variables a, b, c y d.

```
a = 10
b = -5
c = "Hola "
d = [1, 2, 3]

print(a * 5)
print(a - b)
print(c + "Mundo")
print(c * 2)
print(c[-1])
print(c[1:])
print(d + d)
```

```
In [1]: a = 10
b = -5
c = "Hola "
d = [1, 2, 3]

print('a = ' + str(a))
print(type(a))
print('b = ' + str(b))
print(type(b))
print('c = ' + str(c))
print(type(c))
print('d = ' + str(d))
print(type(d))

print('a * 5 = ' + str(a * 5))
print('a - b = ' + str(a - b))
print('c + "Mundo" = ' + str(c + "Mundo"))
print('c * 2 = ' + str(c * 2))
print('c[-1] = ' + str(c[-1]))
print('c[1:] = ' + str(c[1:]))
print('d + d = ' + str(d + d))

a = 10
<class 'int'>
b = -5
<class 'int'>
c = Hola
<class 'str'>
d = [1, 2, 3]
<class 'list'>
a * 5 = 50
a - b = 15
c + "Mundo" = Hola Mundo
c * 2 = Hola Hola
c[-1] = 
c[1:] = ola
d + d = [1, 2, 3, 1, 2, 3]
```

### Ejercicio 2

Escribir un programa que realice la siguiente operación aritmética  $\left(\frac{3+2}{2*5}\right)^2$

```
In [2]: res = ((3+2)/(2*5))**2
res
```

Out[2]: 0.25

### Ejercicio 3

El siguiente código pretende realizar una media entre 3 números, pero no funciona correctamente. ¿Eres capaz de identificar el problema y solucionarlo?

```
In [3]: numero_1 = 9
numero_2 = 3
numero_3 = 6

media = numero_1 + numero_2 + numero_3 / 3
print("La nota media es", media)

La nota media es 14.0
```

```
In [4]: # faltaba un paréntesis en la fórmula

media = (numero_1 + numero_2 + numero_3)/ 3
print("La nota media es", media)
```

La nota media es 6.0

#### Ejercicio 4

Escribir un programa que lea un entero positivo,  $n$ , introducido por el usuario y después muestre en pantalla la suma de todos los enteros desde 1 hasta  $n$ . La suma de los  $n$  primeros enteros positivos puede ser calculada de la siguiente forma:

$$\text{suma} = \frac{n(n+1)}{2}$$

Introduzca un número entero: 5

La suma de los primeros números enteros desde 1 hasta 5 es 15.0

```
In [5]: n = int(input("Introduce un número entero: "))
suma = n * (n + 1) / 2
print("La suma de los primeros números enteros desde 1 hasta " + str(n) + " es " + str(suma))
```

Introduce un número entero: 5

La suma de los primeros números enteros desde 1 hasta 5 es 15.0

#### Ejercicio 5

Queremos calcular el promedio final ponderado de 3 notas y mostrarlo en pantalla. Estos son los pesos:

- La primera nota vale un 15% del total
- La segunda nota vale un 35% del total
- La tercera nota vale un 50% del total

Desarrolla un código para calcular correctamente la nota final, teniendo en cuenta que las notas de un alumno son 18, 14 y 12.

```
In [6]: nota_1 = 18
nota_2 = 14
nota_3 = 10

# Completa el ejercicio aquí
media = (nota_1*0.15 + nota_2*0.35 + nota_3*0.5)
print("La nota media ponderada es", media)
```

La nota media ponderada es 12.6

#### Ejercicio 6

Calcular la siguiente operación  $111...111(50 \text{ veces}) + 222...222(25 \text{ veces}) 333...333(30 \text{ veces})^*$ . Utilice lo revisado en clase.

```
In [7]: int('1' * 50) + int('2' * 25) * int('3' * 30)

Out[7]: 740751851851851851851851777777037037037037037037037
```

#### Ejercicio 7

Imagina que acabas de abrir una nueva cuenta de ahorros que te ofrece el 4% de interés al año. Estos ahorros debido a intereses, que no se cobran hasta finales de año, se te añaden al balance final de tu cuenta de ahorros. Escribir un programa que comience leyendo la cantidad de dinero depositada en la cuenta de ahorros, introducida por el usuario. Después el programa debe calcular y mostrar por pantalla la cantidad de ahorros tras el primer, segundo y tercer años. Redondear cada cantidad a dos decimales con la función `round()`.

Introduce la inversión inicial: 1000

Balance tras el primer año: 1040.0

Balance tras el segundo año: 1081.6

Balance tras el tercer año: 1124.86

```
In [8]: monto = float(input("Introduce la inversión inicial: "))
monto = monto * (1.04)
print("Balance tras el primer año: ", round(monto,2))
monto = monto * (1.04)
print("Balance tras el segundo año: ", round(monto,2))
monto = monto * (1.04)
print("Balance tras el tercer año: ", round(monto,2))
```

Introduce la inversión inicial: 1000

Balance tras el primer año: 1040.0

Balance tras el segundo año: 1081.6

Balance tras el tercer año: 1124.86

#### Ejercicio 8

Solicitar por pantalla la cantidad de segundos a procesar y escribir su representación en horas, minutos y segundos. Como ayuda puede utilizar la división entera (`//`). Por ejemplo:

- Ingrese los segundos: 140
- 140 segundos equivalen a 0 horas, 2 minutos y 20 segundos.

```
In [9]: try:
segundos = int(input("Ingrese los segundos: "))
print(segundos, 'segundos equivalen a', segundos//3600, "horas", (segundos%3600//60), "minutos", segundos%60, "segundos")
except:
print('Error!! Usted debe ingresar un número')
```

Ingrese los segundos: 140  
140 segundos equivalen a 0 horas 2 minutos 20 segundos

## Parte 2: Operaciones con Strings

### Ejercicio 1

Escribir un programa que pregunte el nombre del usuario en la consola y después de que el usuario lo introduzca muestre por pantalla el nombre en mayúsculas y el número de caracteres que contiene en líneas distintas. Por ejemplo:

¿Cómo te llamas? Felipe  
FELIPE tiene 6 letras

```
In [10]: nombre = input('¿Cómo te llamas?')
print(nombre.upper(), 'tiene', len(nombre), 'letras.')
```

¿Cómo te llamas?Luis Garayar  
LUIS GARAYAR tiene 12 letras.

### Ejercicio 2

Dado el siguiente texto, limpiar todo signo de interrogación y puntuación. Finalmente convertir a minúscula y mostrarlo en pantalla. Además de ello mencione el largo de la cadena de caracteres.

```
In [11]: t = '¿En dónde trabajas?, ¿CUÁNTA EXPERIENCIA TIENES?, ¿Por qué has tenido 5 trabajos en solo 2 años?.'
```

```
print(t.replace('¿', '').replace('?', '').replace(',', '').replace('.', '').lower())
print('Largo de la cadena:', len(t))
```

en dónde trabajas cuánta experiencia tienes por qué has tenido 5 trabajos en solo 2 años  
Largo de la cadena: 97

### Ejercicio 3

Ejecute el siguiente código y diga con sus palabras que es lo que hace. Ingrese su respuesta como un comentario en el código.

```
In [12]: str_ex2="Curso_DMC"
str_ex2[str_ex2.index("_")+1:]

# Devuelve La parte de una cadena que va después del guión bajo.
```

Out[12]: 'DMC'

### Ejercicio 4

Al realizar una consulta en un registro hemos obtenido una cadena de texto corrupta al revés. Al parecer contiene el nombre de un alumno y la nota de un examen. ¿Cómo podríamos formatear la cadena y conseguir una estructura como la siguiente?:

- **Nombre Apellido** ha sacado un **Nota** de nota.

Ayuda: Para voltear una cadena rápidamente utilizando slicing podemos utilizar un tercer índice -1: cadena[::-1]

```
In [13]: cadena = "zeréP epeP,81"

cadena2 = cadena[::-1]
print(cadena2[3:], "ha sacado un", cadena2[0:2], "de nota.")
```

Pepe Pérez ha sacado un 18 de nota.

### Ejercicio 5

Supongamos que tenemos los siguientes datos:

- **nombre:** Nataly Silva
- **edad:** 18
- **direccion:** Av. República de Panamá 3055
- **pais:** Perú
- **ingreso:** 3500
- **gastos:** 1820.50

Escribe código para que, tras guardar los datos en variables, se imprima por pantalla:

'Nataly Silva, de 18 años y domicilio en Av. República de Panamá 3055 Perú, tiene restante de su ingreso tras los gastos 1679.50'

```
In [14]: nombre = 'Nataly Silva'
edad = 18
direccion = 'Av. República de Panamá 3055'
pais = 'Perú'
ingreso = 3500
gastos = 1820.50

print(f"{nombre}, de {edad} años y domicilio en {direccion} {pais}, "
      f"tiene restante de su ingreso tras los gastos {ingreso-gastos}")
```

Nataly Silva, de 18 años y domicilio en Av. República de Panamá 3055 Perú, tiene restante de su ingreso tras los gastos 1679.5

## Ejercicio 6

Vamos a crear un pequeño chatbot similar al revisado en clase, pero vamos a hacerlo un poco más largo. Debe empezar haciendo preguntas que debemos ingresar por consola y debe tener una salida similar a la siguiente pero con sus propias respuestas:

**Ingresa tu nombre:** Luis

**Hola Luis, cuál es tu apellido:** Garayar

**Mucho gusto Luis Garayar, donde trabajas:** BCP

**Que buen lugar para trabajar es BCP.**

**Y actualmente donde estudias:** DMC

**Me han contado que enseñan bien en DMC.**

**Que bueno conocer de ti, Luis Garayar.**

**¡Hasta luego!**

```
In [15]: nombre = input("Ingresa tu nombre: ")
apellido = input(f"Hola {nombre}, cuál es tu apellido: ")
trabajo = input(f"Mucho gusto {nombre} {apellido}, ¿donde trabajas?: ")
print(f"Que buen lugar para trabajar es {trabajo}.")
estudio = input("Y actualmente donde estudias: ")
print(f"Me han contado que enseñan bien en {estudio}.")
print(f"Que bueno conocer de ti, {nombre} {apellido}.")
print("¡Hasta luego!")
```

Ingresa tu nombre: Luis  
Hola Luis, cuál es tu apellido: Garayar  
Mucho gusto Luis Garayar, ¿donde trabajas?: BCP  
Que buen lugar para trabajar es BCP.  
Y actualmente donde estudias: DMC  
Me han contado que enseñan bien en DMC.  
Que bueno conocer de ti, Luis Garayar.  
¡Hasta luego!

## Parte 3: Manejo de fechas

### Ejercicio 1

Convierta a formato fecha las siguientes cadenas: (utilice la función **strptime**):

'18/02/2019'

'01/31/2000 12:00:00AM'

```
In [16]: import datetime as dt

datetime_1= dt.datetime.strptime('18/02/2019', '%d/%m/%Y')
datetime_2= dt.datetime.strptime('01/31/2000 12:00:00AM', '%m/%d/%Y %H:%M:%S%p')

print(datetime_1)
print(datetime_2)

2019-02-18 00:00:00
2000-01-31 12:00:00
```

### Ejercicio 2

- ¿Qué día de la semana fue el día que naciste?
- ¿Cuántos días han pasado desde ese día al día de hoy?
- ¿Qué día de la semana será tu cumpleaños dentro de un año?

```
In [17]: import datetime as dt
import locale

locale.setlocale(locale.LC_ALL, ("es_ES", "UTF-8"))

# Parte a
fecha = dt.datetime(1988,4,29)
print('Día de la semana de mi cumpleaños:', fecha.strftime('%A'))

# Parte b
diferencia = dt.datetime.now() - dt.datetime(1988,4,29)
print('Cantidad de días que han pasado desde mi nacimiento:', diferencia.days, 'días')

# Parte c
fecha_sgte = dt.datetime.now() + dt.timedelta(days=365)
print('Día de la semana de mi cumpleaños dentro de un año:', fecha_sgte.strftime('%A'))
```

Día de la semana de mi cumpleaños: viernes  
Cantidad de días que han pasado desde mi nacimiento: 12210 días  
Día de la semana de mi cumpleaños dentro de un año: lunes

```
In [18]: # Volvemos a la configuración local por defecto
locale.setlocale(locale.LC_ALL, ("en_US"))
```

```
Out[18]: 'en_US'
```

### Ejercicio 3

Averigua qué fecha es 10,000 días después del día de hoy. De ese día extrae sus componentes (año, mes, día, día de la semana) y adicionalmente muestra el siguiente mensaje:

**Dentro de 10000 días será el día lunes XX de XXXXX del año XXXX**

```
In [19]: import datetime as dt
import locale

locale.setlocale(locale.LC_ALL, ("es_ES", "UTF-8"))

fecha_hoy = dt.datetime.now()
fecha = fecha_hoy + dt.timedelta(days=10000)
print('Fecha actual:', fecha_hoy.strftime('%d/%m/%Y'))
print('Fecha a futuro:', fecha.strftime('%d/%m/%Y'))
print('Año:', fecha.strftime('%Y'))
print('Mes:', fecha.strftime('%m'))
print('Mes en texto:', fecha.strftime('%B'))
print('Día:', fecha.strftime('%d'))
print('Día de la semana:', fecha.strftime('%A'))

print('Dentro de 10000 días será el día', fecha.strftime('%A'), fecha.strftime('%d'), 'de',
      fecha.strftime('%B'), 'del año', fecha.strftime('%Y'))
```

Fecha actual: 03/10/2021  
Fecha a futuro: 18/02/2049  
Año: 2049  
Mes: 02  
Mes en texto: febrero  
Día: 18  
Día de la semana: jueves  
Dentro de 10000 días será el día jueves 18 de febrero del año 2049

```
In [20]: # Volvemos a la configuración local por defecto
locale.setlocale(locale.LC_ALL, ("en_US"))
```

```
Out[20]: 'en_US'
```

### Ejercicio 4

Realiza el ejercicio anterior, pero que permita ingresar la cantidad de días (el valor de 10,000) por pantalla (usando **input**)

```
In [21]: import datetime as dt
import locale

locale.setlocale(locale.LC_ALL, ("es_ES", "UTF-8"))

dias = int(input('Ingrese la cantidad de días a para la fecha a futuro:'))
fecha_hoy = dt.datetime.now()
fecha = fecha_hoy + dt.timedelta(days=dias)
print('Fecha actual:', fecha_hoy.strftime('%d/%m/%Y'))
print('Fecha a futuro:', fecha.strftime('%d/%m/%Y'))
print('Año:', fecha.strftime('%Y'))
print('Mes:', fecha.strftime('%m'))
print('Mes en texto:', fecha.strftime('%B'))
print('Día:', fecha.strftime('%d'))
print('Día de la semana:', fecha.strftime('%A'))

print('Dentro de', dias, 'días será el día', fecha.strftime('%A'), fecha.strftime('%d'), 'de',
      fecha.strftime('%B'), 'del año', fecha.strftime('%Y'))
```

Ingrese la cantidad de días a para la fecha a futuro:50000  
Fecha actual: 03/10/2021  
Fecha a futuro: 26/08/2158  
Año: 2158  
Mes: 08  
Mes en texto: agosto  
Día: 26  
Día de la semana: sábado  
Dentro de 50000 días será el día sábado 26 de agosto del año 2158

```
In [22]: # Volvemos a La configuración Local por defecto
locale.setlocale(locale.LC_ALL, ("en_US"))
```

```
Out[22]: 'en_US'
```

## Parte 4: Estructuras de control

### Ejercicio 1

Escribir un programa que pida al usuario dos números y muestre por pantalla su división. Si el divisor es cero el programa debe mostrar un mensaje de error usando un condicional.

```
In [23]: d1 = float(input("Ingrese dividendo:"))
d2 = float(input("Ingrese divisor:"))
if d2 != 0:
    res = d1/d2
    print("El resultado es:", res)
else:
    print("El divisor debe ser diferente de cero.")
```

Ingrese dividendo:25  
Ingrese divisor:7  
El resultado es: 3.5714285714285716

### Ejercicio 2

- Escriba una sentencia for para calcular la suma de todos los números entre el 1 y el 100. Es decir, 1+2+3+....+99+100
- Escriba una sentencia para calcular la suma de todos los números pares entre el 1 y el 100, y luego otra para calcular la suma de los números impares.

```
In [24]: # Parte a
suma = 0
for i in range(101):
    suma += i
print(suma)
```

5050

```
In [25]: # Parte b
# Suma de Los números pares
suma_pares = 0
for i in range(101):
    if i % 2 == 0:
        suma_pares += i
print(suma_pares)
```

2550

```
In [26]: # Suma de Los números impares
suma_impar = 0
for i in range(101):
    if i % 2 != 0:
        suma_impar += i
print(suma_impar)
```

2500

### Ejercicio 3

Escribe un programa que solicite por pantalla un número entero (si es otro tipo de valor que muestre un mensaje de error), y que devuelva impreso por pantalla la tabla de multiplicar de dicho número del 1 al 10, por ejemplo:

Ingrese un número: 9

- 9 x 1 = 9
- 9 x 2 = 18
- 9 x 3 = 27
- 9 x 4 = 36
- 9 x 5 = 45
- 9 x 6 = 54
- 9 x 7 = 63
- 9 x 8 = 72
- 9 x 9 = 81
- 9 x 10 = 90

```
In [27]: try:
          num = int(input("Ingrese un número: "))

          for i in range(1,11):
              mult=num*i
              print('-',num, 'x',i,'=',mult)
          except:
              print('Error!.Usted debe ingresar un número entero')
```

```
Ingrese un número: 9
- 9 x 1 = 9
- 9 x 2 = 18
- 9 x 3 = 27
- 9 x 4 = 36
- 9 x 5 = 45
- 9 x 6 = 54
- 9 x 7 = 63
- 9 x 8 = 72
- 9 x 9 = 81
- 9 x 10 = 90
```

### Ejercicio 4

Escribir un programa que almacene la cadena de caracteres **PASSWORD** en una variable, pregunte al usuario por la contraseña hasta que introduzca la contraseña correcta sin tener en cuenta mayúsculas y minúsculas.

Introduce la contraseña: 123456

Introduce la contraseña: Contraseña

Introduce la contraseña: ABC

Introduce la contraseña: PassWORD

Contraseña correcta

```
In [28]: key = "PASSWORD"
          clave = input("Ingrese una contraseña:")
          while (key != clave.upper()):
              clave = input("INCORRECTO!!! Ingrese una contraseña:")
          print("Contraseña ingresada correctamente.")
```

```
Ingrese una contraseña:123456
INCORRECTO!!! Ingrese una contraseña:contraseña
INCORRECTO!!! Ingrese una contraseña:ABCDEF
INCORRECTO!!! Ingrese una contraseña:Password
Contraseña ingresada correctamente.
```

### Ejercicio 5

Escribe un programa que solicite por pantalla la cantidad de notas a ingresar (debe ser un número entero), luego que solicite cada una de esas notas (las notas deben ir del 0 al 20) y calcule el promedio (redondeado a 2 decimales). Si la nota ingresada es menor de 0 o mayor a 20, se debe solicitar ingresar esa nota nuevamente hasta que este correctamente ingresada.

Ingrese la cantidad de valores a ingresar: 4

Ingrese el número 1: 12.5

Ingrese el número 2: 15

Ingrese el número 3: 10

Ingrese el número 4: 13

El promedio de los números ingresados es: 12.62

```
In [29]: try:
x=int(input("Ingrese la cantidad de valores a ingresar:"))
total=0
for i in range(1,x+1):
    y=float(input("Ingrese el número "+str(i)+" :"))
    while ((y>20) | (y<0)):
        print("Las notas deben ser del 0 al 20. Vuelva a intentarlo.")
        y=float(input("Ingrese el número "+str(i)+" :"))
    total=total+y
promedio=total/x
print("EL promedio de Los números ingresados es: ",round(promedio,2))
except:
    print('Error!.Usted debe ingresar un número entero')
```

```
Ingrese la cantidad de valores a ingresar:4
Ingrese el número 1 :12.5
Ingrese el número 2 :15
Ingrese el número 3 :10
Ingrese el número 4 :13
EL promedio de Los números ingresados es: 12.62
```

## Ejercicio 6

Escribir un programa que calcule el monto final (préstamo + interés) a pagar considerando:

- Solicitar el monto del préstamo **M**.
- Si se trata de un **colaborador** del banco, la tasa es del 10.5%
- Si el cliente es del **segmento VIP** la tasa será del 13% si tiene **cuenta de ahorro**, de lo contrario la tasa es del 16%
- En otros casos, la tasa es del 18.5%

```
In [30]: try:
M = float(input("Ingrese el monto de préstamo a 12 meses:"))
S = input("¿A que segmento pertenece?(CLASICO/VIP/OTRO):")
C = input("¿Es colaborador?(SI/NO):")
A = input("¿Tiene cuenta de ahorro?(SI/NO):")
if C.Lower() == 'si':
    I = 0.105
elif S.Lower() == 'vip':
    I = 0.13
elif A.Lower() == 'si':
    I = 0.16
else:
    I = 0.185
print(f"EL monto solicitado es {M} y La TEA es {I*100}%, y el monto anual a pagar es {M*(1+I)}. ")
except:
    print('Error!.Usted debe ingresar un monto numérico')
```

```
Ingrese el monto de préstamo a 12 meses:15000
¿A que segmento pertenece?(CLASICO/VIP/OTRO):clasico
¿Es colaborador?(SI/NO):si
¿Tiene cuenta de ahorro?(SI/NO):no
El monto solicitado es 15000.0 y La TEA es 10.5%, y el monto anual a pagar es 16575.0.
```

## Ejercicio 7

Dado un conjunto de palabras ingresadas por consola, calcule el número de strings que tienen una longitud de 3 o más caracteres y cuyo primer y último caracter sean iguales.

```
In [31]: try:
N = int(input("Ingrese el número de palabras a ingresar:"))
lista = []
cumple = 0
for i in range(N):
    lista.append(input(f"Ingrese la palabra {i+1}:"))
    if len(lista[i]) >= 3 and lista[i][0].Lower() == lista[i][-1].Lower():
        cumple = cumple + 1
print(f"De las {N} palabras, {cumple} tienen 3 o mas letras, y su primera y última letra son iguales")
except:
    print('Error!.Usted debe ingresar un número entero')
```

```
Ingrese el número de palabras a ingresar:5
Ingrese la palabra 1:hola
Ingrese la palabra 2:anita
Ingrese la palabra 3:Kayak
Ingrese la palabra 4:perro
Ingrese la palabra 5:avianca
De las 5 palabras, 3 tienen 3 o mas letras, y su primera y última letra son iguales
```

## Parte 5: Estructuras de datos en Python



### Ejercicio 1

a. Escribir un programa que pregunte al usuario 6 números, los almacene en una lista y los muestre por pantalla ordenados de menor a mayor.

- Introduce un número: 46
- Introduce un número: 25
- Introduce un número: 32
- Introduce un número: 15
- Introduce un número: 9
- Introduce un número: 21
- Los números ingresados son [9, 15, 21, 25, 32, 46]

b. Cree un comando que mueva los dos últimos elementos de la lista al comienzo de la misma (Ayuda: puede utilizar las funciones de listas remove e insert)

```
In [32]: # Parte a
lista = []
for i in range(1,7):
    lista.append(int(input("Ingresa el número " + str(i) + " : ")))
lista.sort()
print("Los números ingresados son:", lista)
```

```
Ingresa el número 1 : 46
Ingresa el número 2 : 25
Ingresa el número 3 : 32
Ingresa el número 4 : 15
Ingresa el número 5 : 9
Ingresa el número 6 : 21
Los números ingresados son: [9, 15, 21, 25, 32, 46]
```

```
In [33]: #Parte b
number1=lista[-1]
number2=lista[-2]
lista.remove(number1)
lista.remove(number2)
lista.insert(0,number1)
lista.insert(0,number2)

print("El listado con los elementos reordenados es:", lista)
```

```
El listado con los elementos reordenados es: [32, 46, 9, 15, 21, 25]
```

### Ejercicio 2

Genere una lista de números enteros del 1 al 99 (usando list y range) y elimine de la lista los múltiplos de 5 y de 7. Finalmente muestre el listado los números restantes.

```
In [34]: numeros = list(range(1,100))
for num in numeros:
    if num % 5 == 0:
        numeros.remove(num)
for num in numeros:
    if num % 7 == 0:
        numeros.remove(num)
print(numeros)
```

```
[1, 2, 3, 4, 6, 8, 9, 11, 12, 13, 16, 17, 18, 19, 22, 23, 24, 26, 27, 29, 31, 32, 33, 34, 36, 37, 38, 39, 41, 43, 44, 46, 47, 48, 5
1, 52, 53, 54, 57, 58, 59, 61, 62, 64, 66, 67, 68, 69, 71, 72, 73, 74, 76, 78, 79, 81, 82, 83, 86, 87, 88, 89, 92, 93, 94, 96, 97, 9
9]
```

### Ejercicio 3

Escribir un programa que almacene las asignaturas de un curso (Matemáticas, Física, Química, Historia y Lenguaje) en una lista, pregunte al usuario la nota que ha sacado en cada asignatura y elimine de la lista las asignaturas aprobadas. Al final el programa debe mostrar por pantalla las asignaturas que el usuario tiene que repetir.

- ¿Qué nota has sacado en Matemáticas? 14
- ¿Qué nota has sacado en Física? 8
- ¿Qué nota has sacado en Química? 11
- ¿Qué nota has sacado en Historia? 12
- ¿Qué nota has sacado en Lenguaje? 6
- Tienes que repetir ['Física', 'Lenguaje']

```
In [35]: # Forma 1
subjects = ["Matemáticas", "Física", "Química", "Historia", "Lengua"]
passed = []
for subject in subjects:
    score = float(input("¿Qué nota has sacado en " + subject + "? "))
    if score >= 11:
        passed.append(subject)
for subject in passed:
    subjects.remove(subject)
print("Tienes que repetir " + str(subjects))
```

```
¿Qué nota has sacado en Matemáticas? 14
¿Qué nota has sacado en Física? 8
¿Qué nota has sacado en Química? 11
¿Qué nota has sacado en Historia? 12
¿Qué nota has sacado en Lengua? 6
Tienes que repetir ['Física', 'Lengua']
```

```
In [36]: # Forma 2
subjects = ["Matemáticas", "Física", "Química", "Historia", "Lengua"]
for i in range(len(subjects)-1, -1, -1):
    score = float(input("¿Qué nota has sacado en " + subjects[i] + "? "))
    if score >= 11:
        subjects.pop(i)
print("Tienes que repetir " + str(subjects))

¿Qué nota has sacado en Lengua? 14
¿Qué nota has sacado en Historia? 8
¿Qué nota has sacado en Química? 11
¿Qué nota has sacado en Física? 9
¿Qué nota has sacado en Matemáticas? 16
Tienes que repetir ['Física', 'Historia']
```

#### Ejercicio 4

Escribir un programa que pida al usuario una palabra (o frase) y muestre por pantalla el número de veces que contiene cada vocal.

- Introduce una palabra: En un lugar de la Mancha
- La vocal a aparece 4 veces
- La vocal e aparece 2 veces
- La vocal i aparece 0 veces
- La vocal o aparece 0 veces
- La vocal u aparece 2 veces

```
In [37]: word = input("Introduce una palabra: ")
vocal = ['a', 'e', 'i', 'o', 'u']
for vocal in vocal:
    times = 0
    for letter in word.lower():
        if letter == vocal:
            times += 1
    print("La vocal " + vocal + " aparece " + str(times) + " veces")

Introduce una palabra: En un lugar de la Mancha
La vocal a aparece 4 veces
La vocal e aparece 2 veces
La vocal i aparece 0 veces
La vocal o aparece 0 veces
La vocal u aparece 2 veces
```

#### Ejercicio 5

Crear los siguientes conjuntos: {a,b,c,d,e,f,g,h,i} y {a,e,i,o,u}, y realice las operaciones de unión, intersección y diferencia de conjuntos. Repita lo anterior con estos conjuntos: {1,2,3,4,5,6,7,8,9} y {1,3,5,7,9}

```
In [38]: conj1 = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i'}
conj2 = {'a', 'e', 'i', 'o', 'u'}

print("Unión:", conj1.union(conj2))
print("Intersección:", conj1.intersection(conj2))
print("Diferencia A-B:", conj1.difference(conj2))
print("Diferencia B-A:", conj2.difference(conj1))
print('-----')

conj3 = {1,2,3,4,5,6,7,8,9}
conj4 = {3,5,7,9}

print("Unión:", conj3.union(conj4))
print("Intersección:", conj3.intersection(conj4))
print("Diferencia A-B:", conj3.difference(conj4))
print("Diferencia B-A:", conj4.difference(conj3))

Unión: {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'o', 'u'}
Intersección: {'a', 'e', 'i'}
Diferencia A-B: {'b', 'c', 'd', 'f', 'g', 'h'}
Diferencia B-A: {'o', 'u'}
-----
Unión: {1, 2, 3, 4, 5, 6, 7, 8, 9}
Intersección: {3, 5, 7}
Diferencia A-B: {1, 2, 4, 6, 8}
Diferencia B-A: set()
```

#### Ejercicio 6

Usando conjuntos y listas, ingrese algunas líneas de código que permita ingresar una palabra, convertirlo en mayúscula, que elimine las vocales y queden las consonantes distintas. Finalmente debe devolver el listado de consonantes en orden descendente.

- Ingrese una palabra: murciélago
- ['R', 'M', 'L', 'G', 'C']

```
In [39]: palabra=input('Ingresa una palabra:')
palabra= set(palabra.upper())
vocales ={'A','Á','E','É','I','Í','O','Ó','U','Ú'}
m=palabra-vocales
m=list(m)
m.sort(reverse = True)
print(m)
```

Ingresa una palabra:otorrinolaringolo  
['T', 'R', 'N', 'L', 'G']

## Ejercicio 7

Escribir un programa que reciba el siguiente texto y muestre:

- Cantidad de palabras distintas
- Frecuencia de cada palabra encontrada

**Buenas tardes, he notado consumos no reconocidos en mi tarjeta de crédito que no he realizado. Mis últimos consumos son del 4 de abril pero en el estado de cuenta de mi tarjeta de crédito aparecen consumos posteriores. Espero su pronta ayuda, abonando el monto de los consumos al saldo de mi tarjeta, gracias.**

```
In [40]: texto = 'Buenas tardes, he notado consumos no reconocidos en mi tarjeta de crédito que no he realizado. Mis últimos consumos son del
4 de abril pero en el estado de cuenta de mi tarjeta de crédito aparecen consumos posteriores. Espero su pronta ayuda, abonando el m
onto de los consumos al saldo de mi tarjeta, gracias.'
lista = texto.replace(' ','').replace(',','').lower().split()
cantidad = len(set(lista))
frecuencia = {}
for w in lista:
    frecuencia[w] = lista.count(w)
frec_orden = sorted(frecuencia.items())
print('Cantidad de palabras distintas: ',cantidad)
print('Frecuencia de cada palabra:')
for item in enumerate(frec_orden):
    if item[1][1] == 1:
        print("\tLa palabra", item[1][0], "se repite", item[1][1], "vez.")
    else:
        print("\tLa palabra", item[1][0], "se repite", item[1][1], "veces.")
```

Cantidad de palabras distintas: 36

Frecuencia de cada palabra:

La palabra 4 se repite 1 vez.  
La palabra abonando se repite 1 vez.  
La palabra abril se repite 1 vez.  
La palabra al se repite 1 vez.  
La palabra aparecen se repite 1 vez.  
La palabra ayuda se repite 1 vez.  
La palabra buenas se repite 1 vez.  
La palabra consumos se repite 4 veces.  
La palabra crédito se repite 2 veces.  
La palabra cuenta se repite 1 vez.  
La palabra de se repite 7 veces.  
La palabra del se repite 1 vez.  
La palabra el se repite 2 veces.  
La palabra en se repite 2 veces.  
La palabra espero se repite 1 vez.  
La palabra estado se repite 1 vez.  
La palabra gracias se repite 1 vez.  
La palabra he se repite 2 veces.  
La palabra los se repite 1 vez.  
La palabra mi se repite 3 veces.  
La palabra mis se repite 1 vez.  
La palabra monto se repite 1 vez.  
La palabra no se repite 2 veces.  
La palabra notado se repite 1 vez.  
La palabra pero se repite 1 vez.  
La palabra posteriores se repite 1 vez.  
La palabra pronta se repite 1 vez.  
La palabra que se repite 1 vez.  
La palabra realizado se repite 1 vez.  
La palabra reconocidos se repite 1 vez.  
La palabra saldo se repite 1 vez.  
La palabra son se repite 1 vez.  
La palabra su se repite 1 vez.  
La palabra tardes se repite 1 vez.  
La palabra tarjeta se repite 3 veces.  
La palabra últimos se repite 1 vez.

## Ejercicio 8

Almacene en un diccionario los precios de las frutas de la siguiente tabla. Luego pregunte al usuario por una fruta, el número de kilos y muestre por pantalla el precio de ese número de kilos de fruta. Si la fruta no está en el diccionario debe mostrar un mensaje informando de ello.

| Fruta   | Precio |
|---------|--------|
| Plátano | 1.35   |
| Manzana | 0.80   |
| Pera    | 0.85   |
| Naranja | 0.70   |

```
In [41]: diccionario_frutas = {'Plátano':1.35, 'Manzana':0.8, 'Pera':0.85, 'Naranja':0.7}

try:
    fruta= input('¿Qué fruta desea?:').title()
    kilos= float(input('Cuántos kilos desea?:'))
    if fruta in diccionario_frutas.keys():
        print('Precio Total:',diccionario_frutas[fruta]*kilos)
    else:
        print('No vendemos',fruta, '. Muchas Gracias')
except:
    print('Ingrese valores numéricos.')
```

```
¿Qué fruta desea?:Uva
Cuántos kilos desea?:10
No vendemos Uva . Muchas Gracias
```

## Ejercicio 9

Escribir un programa que cree un diccionario simulando una cesta de la compra. El programa debe preguntar el artículo y su precio y añadir el par al diccionario, hasta que el usuario decida terminar (use un bucle). Después se debe mostrar por pantalla la lista de la compra y el coste total, con el siguiente formato:

| Lista de la compra |        |
|--------------------|--------|
| Artículo 1         | Precio |
| Artículo 2         | Precio |
| Artículo 3         | Precio |
| ...                | ...    |
| Total              | Coste  |

```
In [42]: canasta = {}
more = 'Si'
while more.title() == 'Si':
    print('-----')
    item = input('Introduce un artículo: ')
    price = float(input('Introduce el precio de ' + item + ': '))
    canasta[item] = price
    more = input('¿Quieres añadir artículos a la lista (Si/No)? ')
cost = 0
print('-----')
print('Lista de la compra')
for item, price in canasta.items():
    print(item, ': ', price)
    cost += price
print('Coste total: ', cost)
```

```
-----
Introduce un artículo: Pintura
Introduce el precio de Pintura: 12
¿Quieres añadir artículos a la lista (Si/No)? Si
-----
Introduce un artículo: Fierro
Introduce el precio de Fierro: 5
¿Quieres añadir artículos a la lista (Si/No)? Si
-----
Introduce un artículo: Tornillos
Introduce el precio de Tornillos: 3
¿Quieres añadir artículos a la lista (Si/No)? No
-----
Lista de la compra
Pintura : 12.0
Fierro : 5.0
Tornillos : 3.0
Coste total: 20.0
```

## Parte 6: Librería NumPy

### Ejercicio 1

Genere el código para obtener los resultados de la siguiente expresión. Puede buscar las funciones necesarias de la librería NumPy en la web para completar este ejercicio de ser necesario.

$$\frac{\sqrt{(40 + 25 * 3.5)}}{\log_{10}(35)} + \left( \frac{40 + 35}{2} \right) * (e^2 + 20)$$

```
In [43]: import numpy as np

print("Resultado:")
print((np.sqrt(40+25*3.5)/np.log10(35))+((40+35)/2)*(np.exp(2)+20))
```

```
Resultado:
1034.4024872527925
```

## Ejercicio 2

- a. Crear un vector con los números enteros entre 10 y 50
- b. Crear un vector con los números enteros impares entre 40 y 80

```
In [44]: import numpy as np

# Parte a
x = np.arange(10,51)
print(x)

print('-----')
# Parte b
y = np.arange(40,81,2)
print(y)

[10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50]
-----
[40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72 74 76 78 80]
```

## Ejercicio 3

Genere las siguientes matrices usando operaciones con arrays.

$$B = \begin{bmatrix} 1 & 12 & 25 & 100 \\ 2 & 13 & 26 & 101 \\ 3 & 14 & 27 & 102 \\ 4 & 15 & 28 & 103 \end{bmatrix}$$

$$D = \begin{bmatrix} 11 & 1 & 1 & 1 & 1 \\ 1 & 21 & 1 & 1 & 1 \\ 1 & 1 & 31 & 1 & 1 \\ 1 & 1 & 1 & 41 & 1 \\ 1 & 1 & 1 & 1 & 51 \end{bmatrix}$$

```
In [45]: import numpy as np

print("Primera matriz:")
print(np.array([range(1,5),range(12,16),range(25,29),range(100,104)]).T)

print("-----")
print("Segunda matriz:")
print(np.ones((5,5)) + np.diag(np.arange(1,6)*10))

Primera matriz:
[[ 1 12 25 100]
 [ 2 13 26 101]
 [ 3 14 27 102]
 [ 4 15 28 103]]
-----
Segunda matriz:
[[11.  1.  1.  1.  1.]
 [ 1. 21.  1.  1.  1.]
 [ 1.  1. 31.  1.  1.]
 [ 1.  1.  1. 41.  1.]
 [ 1.  1.  1.  1. 51.]]
```

## Ejercicio 4

Crear una matriz de 5x5 con valores en las filas que sean los número impares del 1 al 9

Ayuda: Las funciones `tile` y `arange` pueden ayudar

```
In [46]: np.tile(np.arange(1,10,2),(5,1))

Out[46]: array([[1, 3, 5, 7, 9],
 [1, 3, 5, 7, 9],
 [1, 3, 5, 7, 9],
 [1, 3, 5, 7, 9],
 [1, 3, 5, 7, 9]])
```

## Ejercicio 5

Crear 2 matrices cuadradas de números (que no sean ceros) de orden  $m$  (donde  $m \geq 3$ , pero que sean del mismo orden).

a. Realizar las operaciones de suma, resta, multiplicación escalar, multiplicación matricial y división escalar.

b. Escoja cualquiera de las matrices creadas, calcular la transpuesta, la determinante y la inversa de esa matriz. **Nota:** Si la determinante es cero (con lo que la matriz inversa no existe), prube con la otra matriz para la inversa.

```
In [47]: # Parte a
m = np.random.randint(1, 20, (4, 4))
n = np.random.randint(21, 40, (4, 4))

print("La primera matriz es:\n",m)
print("\nLa segunda matriz es:\n",n)
print("\nSuma:\n",m+n)
print("\nResta:\n",m-n)
print("\nMultiplicación escalar:\n",m*n)
print("\nMultiplicación matricial:\n",np.dot(m,n))
print("\nDivisión escalar:\n",m/n)
```

La primera matriz es:

```
[[13 10 17  6]
 [ 6  9  8  4]
 [17  7  5 18]
 [ 9  1 10  7]]
```

La segunda matriz es:

```
[[27 28 35 36]
 [28 32 23 38]
 [26 24 23 24]
 [22 27 39 26]]
```

Suma:

```
[[40 38 52 42]
 [34 41 31 42]
 [43 31 28 42]
 [31 28 49 33]]
```

Resta:

```
[[ -14 -18 -18 -30]
 [ -22 -23 -15 -34]
 [  -9 -17 -18  -6]
 [ -13 -26 -29 -19]]
```

Multipliación escalar:

```
[[351 280 595 216]
 [168 288 184 152]
 [442 168 115 432]
 [198  27 390 182]]
```

Multipliación matricial:

```
[[1205 1254 1310 1412]
 [ 710  756  757  854]
 [1181 1306 1573 1466]
 [ 685  713  841  784]]
```

División escalar:

```
[[0.48148148 0.35714286 0.48571429 0.16666667]
 [0.21428571 0.28125      0.34782609 0.10526316]
 [0.65384615 0.29166667 0.2173913  0.75      ]
 [0.40909091 0.03703704 0.25641026 0.26923077]]
```

```
In [48]: # Parte b
#Matrices transpuestas
print("Transpuesta de la primera matriz:\n", m.T)
print("\nDeterminante:\n", np.linalg.det(m))
print("\nInversa:\n", np.linalg.inv(m))
```

Transpuesta de la primera matriz:

```
[[13  6 17  9]
 [10  9  7  1]
 [17  8  5 10]
 [ 6  4 18  7]]
```

Determinante:

```
-3317.999999999996
```

Inversa:

```
[[ 0.38969259 -0.47830018  0.10578662 -0.33273056]
 [-0.00783605  0.1217601  0.01024714 -0.08921037]
 [-0.1172393  0.16787221 -0.0774563  0.20373719]
 [-0.33242917  0.35774563 -0.02682339  0.29234479]]
```

## Ejercicio 6

Resuelva el siguiente sistema de ecuaciones con el método matricial  $Ax = y$ . El sistema se resuelve realizando lo siguiente:  $x = \text{inversa}(A)y$

$$x_1 + 2x_2 + 3x_3 + 4x_4 + 5x_5 = 7$$

$$2x_1 + x_2 + 2x_3 + 3x_4 + 4x_5 = -1$$

$$3x_1 + 2x_2 + x_3 + 2x_4 + 3x_5 = -3$$

$$4x_1 + 3x_2 + 2x_3 + x_4 + 2x_5 = 5$$

$$5x_1 + 4x_2 + 3x_3 + 2x_4 + x_5 = 17$$

```
In [49]: A = np.array([[1,2,3,4,5],[2,1,2,3,4],[3,2,1,2,3],[4,3,2,1,2],[5,4,3,2,1]])
b = np.array([7,-1,-3,5,17])

print("Matriz de coeficientes A:")
print(A)
print("-----")
print("Vector de resultados b")
print(b)

# ¿La matriz A puede tener inversa? Podemos ver esto con la determinante:
print("-----")
print("Determinante de La matriz A:")
print(round(np.linalg.det(A),1))

print("-----")
print("Resolviendo el sistema de ecuaciones:")
x = np.dot(np.linalg.inv(A),b)
print(x)

print("-----")
print("Comprobando la respuesta (debe salir cero):")
print(np.around(np.dot(A,x) - b,0))
```

```
Matriz de coeficientes A:
[[1 2 3 4 5]
 [2 1 2 3 4]
 [3 2 1 2 3]
 [4 3 2 1 2]
 [5 4 3 2 1]]
-----
Vector de resultados b
[ 7 -1 -3  5 17]
-----
Determinante de la matriz A:
48.0
-----
Resolviendo el sistema de ecuaciones:
[-2.  3.  5.  2. -4.]
-----
Comprobando la respuesta (debe salir cero):
[ 0. -0.  0.  0.  0.]
```

## Parte 7: Funciones

### Ejercicio 1

Crear una función de devuelva el área de un triángulo o de un rectángulo ingresando 3 parámetros: la base, la altura y el número de lados: 3 o 4. Si es un valor distinto o de otro tipo, arrojar un mensaje de error (recuerde el manejo de errores)

```
In [50]: def area_forma(x,y,z):
    if z==3:
        print('Es un triángulo')
        area= x*y/2
        print('EL valor del área es:',area)
    elif z==4:
        print('Es un Rectángulo')
        area= x*y
        print('EL valor del área es:',area)
    else:
        print('No es un triángulo ni un rectángulo')

area_forma(3,4,4)

Es un Rectángulo
El valor del área es: 12
```

### Ejercicio 2

Crear una función que calcule el interes simple generado por un capital C, en un tiempo T (dado en meses) bajo una tasa R (anual).

```
In [51]: C = float(input('Capital: '))
T = float(input('Tiempo en meses: '))
R = float(input('Interes anual en %: '))

def iSimple(C,T,R):
    M = R/1200 # pasamos el interés anual a mensual
    return C * T * M

print('\nMonto a pagar:',iSimple(C,T,R)+C)
print('Interes simple:',iSimple(C,T,R))

Capital: 1000
Tiempo en meses: 12
Interes anual en %: 25

Monto a pagar: 1250.0
Interes simple: 250.0
```

### Ejercicio 3

Crear una función que calcule el promedio iterativo de una lista numérica. Por ejemplo: para la lista `range(10)` se obtendrá el siguiente resultado:

La media en la iteración 1 es 0.0  
La media en la iteración 2 es 0.5  
La media en la iteración 3 es 1.0  
La media en la iteración 4 es 1.5  
La media en la iteración 5 es 2.0  
La media en la iteración 6 es 2.5  
La media en la iteración 7 es 3.0  
La media en la iteración 8 es 3.5  
La media en la iteración 9 es 4.0  
La media en la iteración 10 es 4.5

```
In [52]: # el input de la función tiene que ser una Lista numérica

def promedio_iterativo(Lista):
    suma = 0
    elementos = 0
    for i in range(0, len(Lista)):
        suma = suma + Lista[i]
        elementos = elementos + 1
        print('La media en la iteración '+str(elementos)+' es "+str(suma/elementos))

promedio_iterativo(List(range(10)))

La media en la iteración 1 es 0.0
La media en la iteración 2 es 0.5
La media en la iteración 3 es 1.0
La media en la iteración 4 es 1.5
La media en la iteración 5 es 2.0
La media en la iteración 6 es 2.5
La media en la iteración 7 es 3.0
La media en la iteración 8 es 3.5
La media en la iteración 9 es 4.0
La media en la iteración 10 es 4.5
```

### Ejercicio 4

La secuencia de Fibonacci es una secuencia en matemática que comienza con 0 y 1, y luego cada entrada sucesiva es la suma de los dos anteriores. Por lo tanto, la secuencia va 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ... (lo vimos en los ejemplos de clase). Vamos a hacer una pequeña modificación...

Crear una función que reciba como parámetros un número entero  $n$  mayor que 0, además de 2 valores de inicio  $a$  y  $b$ , y calcule la secuencia modificada de Fibonacci de  $n$  elementos. Use control iterativo for

**Fib[0] = a, Fib[1] = b, Fib[n] = Fib[n-1] + Fib[n-2] <math>b>**

**Nota:** Controle los errores para que salga un mensaje si se ingresa algo distinto a un valor numérico.

```
In [53]: def fibonacci(n,a,b):
    sequence = [a,b]
    for i in range(2,n): # Para todo n<=2
        sequence.append(sequence[i-1]+sequence[i-2])
    return sequence

fib = fibonacci(10,1,2)
print(fib)

[1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

### Ejercicio 5

Cree una función llamada `potencia_recursivo` que reciba 2 parámetros ( $b$ ,  $n$ ) y realice el resultado de  $b$  a la potencia de  $n$  pero de manera recursiva (multiplicando) ( $b$  y  $n$  deben ser valores numéricos)

```
In [54]: def potencia_recursivo(b,n):
    if n>1:
        b = b * potencia_recursivo(b,n-1)
        return b
    if n==1:
        b = b * 1
        return b
    if n<1:
        print ('Error!.Debes ingresar un número entero positivo')
```

```
In [55]: potencia_recursivo(5,4)
```

```
Out[55]: 625
```

### Ejercicio 6

La secuencia de Fibonacci es una secuencia en matemática que comienza con 0 y 1, y luego cada entrada sucesiva es la suma de los dos anteriores. Por lo tanto, la secuencia va 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ... (lo vimos en los ejemplos en clase)

Implemente la serie de Fibonacci de un número usando una función recursiva.



```
In [56]: def recur_fibo(n):
        if n <= 1:
            return n
        else:
            return(recur_fibo(n-1) + recur_fibo(n-2))

nterms = 10

# mostramos la secuencia
if nterms <= 0:
    print("Por favor ingrese un número entero positivo")
else:
    print("Secuencia de Fibonacci:")
    for i in range(nterms):
        print(recur_fibo(i))
```

```
Secuencia de Fibonacci:
0
1
1
2
3
5
8
13
21
34
```

### Ejercicio 7

Crear una función que simule un sistema de bloqueo de un celular mediante un pin (ese pin es 1234). Debe permitir realizar 3 (ingresados por pantalla con la función input) intentos antes de mostrar un mensaje de pin incorrecto y acabar el programa.

```
In [57]: def bloqueo():
        pin = 1234
        intento = 0
        while intento < 3:
            ingresado = int(input("Ingrese el PIN de desbloqueo: "))
            if ingresado == pin:
                print("Celular desbloqueado.")
                break
            else:
                print( str(intento+1) + " intento")
                intento += 1
        if intento == 3:
            print("Intentos agotados.")
        bloqueo()
```

```
Ingrese el PIN de desbloqueo: 1111
1 intento
Ingrese el PIN de desbloqueo: 7894
2 intento
Ingrese el PIN de desbloqueo: 1234
Celular desbloqueado.
```

### Ejercicio 8

Cree una función llamada resta la cual permita restar dos valores que son ingresados como parámetros. El primer parámetro se llama x y es obligatorio, mientras que el segundo es opcional y por defecto tiene un valor igual a 0. Evalúe la función de dos maneras: usando solo un parámetro y luego usando ambos parámetros de la función

```
In [63]: def resta(x,y=0):
        r = x-y
        return r
```

```
In [64]: print(resta(10))
        print(resta(10,3))
```

```
10
7
```

### Ejercicio 9

Cree una función llamada prod\_numeros la cual permita multiplicar una cantidad indefinida de valores que son ingresados como parámetros. El primer parámetro se llama "a", es obligatorio y tiene el valor de uno por defecto, mientras que los demás son opcionales e indefinidos. Evalúe esta función sin parámetros, con un parámetro y con varios parámetros (no debe arrojar error en ninguno de los 3 casos)

```
In [59]: def prod_numeros(*valor,a=1):
        s=1
        for argumento in valor:
            s=argumento*s*a
        return s
```

```
In [60]: print(prod_numeros())
        print(prod_numeros(10))
        print(prod_numeros(10,5,6,2,5,9))
```

```
1
10
27000
```

## Ejercicio 10

Implemente un módulo en python (en formato .py) que tenga estas funciones:

- Sumar los primeros N número naturales.
- Consultar si un número es primo o no.
- Calcular el factorial de un número.
- Agregar una función adicional creada por usted...

Agrégueme los comentarios respectivos y pruebelos con la función help. Importe el módulo y realice una operación con cada función (No olvidar enviar el módulo junto con este notebook para poder revisarlo)

```
In [ ]: # contenido del modulo mi_primer_modulo.py
# se muestran las funciones principales que debe contener este módulo

'''--- Ejercicio 10 ---'''

# Sumar Los primeros N número naturales.
def sumar_numeros_naturales(n):
    suma=0
    for i in range(1,n+1):
        suma=suma+i
    return suma

# Consultar si un número es primo o no.
import math
def numero_primo(n):
    if (n<=1):
        return False
    for i in range(2, math.ceil(math.sqrt(n))+1):
        if(n%i==0 and i!=n):
            return False
    return True

# Calcular el factorial de un número.
def factorial(n):
    if n==0 or n==1:
        resultado=1
    else:
        resultado=n*factorial(n-1)
    return resultado
```

```
In [65]: import mi_primer_modulo as modulo

help(modulo)

print(modulo.sumar_numeros_naturales(10))
print(modulo.numero_primo(12))
print(modulo.factorial(5))

Help on module mi_primer_modulo:

NAME
    mi_primer_modulo - --- Ejercicio 10 ---

FUNCTIONS
    factorial(n)
        # Calcular el factorial de un número.

    numero_primo(n)

    sumar_numeros_naturales(n)
        # Sumar Los primeros N número naturales.

FILE
    c:\users\luis_felipe\dmc_python\aa_analytics_python\mi_primer_modulo.py

55
False
120
```

## Ejercicio 11

Diseña un módulo en python (en formato .py) que facilite el trabajo con listas como si fueran conjuntos (sets). Se deberá implementar las siguientes funciones:

- lista\_a\_conjunto(lista): Devuelve un conjunto con los mismos elementos que hay en lista, pero sin repeticiones. (Ejemplo: lista\_a\_conjunto [1,1,3,2,3] devolverá la lista [1,2,3], aunque también se acepta como equivalente cualquier permutación de esos mismos elementos, como [3,1,2] o [3,2,1]).
- unión(lista A, lista B): devuelve el conjunto resultante de unir las listas A y B.
- intersección(lista A, lista B): devuelve el conjunto cuyos elementos pertenecen a las listas A y a B.
- diferencia(lista A, lista B): devuelve el conjunto de elementos que pertenecen a las listas A y no a B.
- iguales(A lista A, lista B): devuelve cierto si A y B tienen los mismos elementos y falso en caso contrario. (Nota: ten en cuenta que los conjuntos representados por las listas [1,3,2] y [2,1,3] son iguales).

Agrégueme los comentarios respectivos y pruebelos con la función help. Importe el módulo y realice una operación con cada función (No olvidar enviar el módulo junto con este notebook para poder revisarlo)

```
In [ ]: # contenido del modulo modulo_conjuntos.py
# se muestran Las funciones principales que debe contener este módulo

'''
--- Ejercicio 11 ---
Módulo que permite el trabajo con Listas como si fueran conjuntos (sets).
Recuerda que un conjunto es una Lista en La que no hay elementos repetidos.
'''

# Devuelve un conjunto con Los mismos elementos que hay en lista, pero sin repeticiones
def lista_a_conjunto(Lista):
    return(List(set(Lista)))

# Devuelve el conjunto resultante de unir Las Listas A y B.
def union(A,B):
    conj_A = set(A)
    conj_B = set(B)
    conj_U = conj_A.union(conj_B)
    return(List(conj_U))

# devuelve el conjunto cuyos elementos pertenecen a Las Listas A y a B.
def interseccion(A,B):
    conj_A = set(A)
    conj_B = set(B)
    conj_U = conj_A.intersection(conj_B)
    return(List(conj_U))

# devuelve el conjunto de elementos que pertenecen a Las Listas A y no a B.
def diferencia(A,B):
    conj_A = set(A)
    conj_B = set(B)
    conj_U = conj_A.difference(conj_B)
    return(List(conj_U))

# devuelve cierto si A y B tienen Los mismos elementos y falso en caso contrario.
def iguales(A,B):
    conj_A = set(A)
    conj_B = set(B)
    res = 'Las Listas son iguales' if conj_A == conj_B else 'Las Listas son diferentes'
    return(res)
```

```
In [66]: import modulo_conjuntos as conj

help(conj)

A = [1,1,3,2,3]
B = [1,3,2]
C = [1,2,4,3,5,6]

print(conj.lista_a_conjunto(A))
print(conj.union(A,C))
print(conj.interseccion(A,C))
print(conj.diferencia(C,A))
print(conj.iguales(A,B))
print(conj.iguales(A,C))

Help on module modulo_conjuntos:

NAME
    modulo_conjuntos

DESCRIPTION
    --- Ejercicio 11 ---
    Módulo que permite el trabajo con Listas como si fueran conjuntos (sets).
    Recuerda que un conjunto es una Lista en La que no hay elementos repetidos.

FUNCTIONS
    diferencia(A, B)
        # devuelve el conjunto de elementos que pertenecen a Las Listas A y no a B.

    iguales(A, B)
        # devuelve cierto si A y B tienen Los mismos elementos y falso en caso contrario.

    interseccion(A, B)
        # devuelve el conjunto cuyos elementos pertenecen a Las Listas A y a B.

    lista_a_conjunto(Lista)
        # Devuelve un conjunto con Los mismos elementos que hay en lista, pero sin repeticiones

    union(A, B)
        # Devuelve el conjunto resultante de unir Las Listas A y B.

FILE
    c:\users\Luis_felipe\dmc_python\aa_analytics_python\modulo_conjuntos.py

[1, 2, 3]
[1, 2, 3, 4, 5, 6]
[1, 2, 3]
[4, 5, 6]
Las listas son iguales
Las listas son diferentes
```

## Ejercicio 12

- Usa comprensión de listas para calcular todos los números pares hasta el 50 inclusive.
- Vuelva a realizar el mismo programa pero con una función lambda (usando list y filter)

```
In [67]: pares = [number for number in range(0,51) if number % 2 == 0]
print(pares)

[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50]

In [68]: pares2 = List(filter(Lambda num: num % 2 == 0, range(0,51)))
print(pares2)

[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50]
```

## Ejercicio 13

Usando la comprensión de listas, cree una lista que contenga los elementos en la secuencia de Fibonacci(n) que sean pares. Reutilice la función de la serie de Fibonacci que revisamos en clase. Pruébalo para n = 10, 20 y 30.

Resultado: [0,2,8,34,...]

```
In [69]: print('Fibonacci:',List(map(recur_fibo,range(10))))

List_fibo = [number for number in List(map(recur_fibo,range(10))) if number % 2 == 0]
print('Fibonacci pares:',list_fibo)

Fibonacci: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
Fibonacci pares: [0, 2, 8, 34]

In [70]: print('Fibonacci:',List(map(recur_fibo,range(20))))

List_fibo = [number for number in List(map(recur_fibo,range(20))) if number % 2 == 0]
print('Fibonacci pares:',list_fibo)

Fibonacci: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181]
Fibonacci pares: [0, 2, 8, 34, 144, 610, 2584]

In [71]: print('Fibonacci:',List(map(recur_fibo,range(30))))

List_fibo = [number for number in List(map(recur_fibo,range(30))) if number % 2 == 0]
print('Fibonacci pares:',list_fibo)

Fibonacci: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, 514229]
Fibonacci pares: [0, 2, 8, 34, 144, 610, 2584, 10946, 46368, 196418]
```

## Ejercicio 14

a. Cree una función lambda que reciba 2 parámetros y que devuelva la suma de los cuadrados de ambos números si el primer número es mayor al segundo, caso contrario que devuelva la suma de los cubos de ambos números. Compruébelo con algunos ejemplos

```
In [72]: resultado = Lambda x,y: x**2 + y**2 if x>y else x**3 + y**3

print(resultado(3,5))
print(resultado(5,3))

152
34
```

b. Evalúe la función lambda anterior para los siguientes vectores (usando list y map).

```
In [73]: W = [3,4,5,6,7]
Y = [9,7,5,3,1]

List(map(resultado,W,Y))

Out[73]: [756, 407, 250, 45, 50]
```

## Ejercicio 15

Cree una función lambda que reciba una cantidad de segundos y lo devuelva en horas, minutos y segundos. Por ejemplo:

tiempo(3895)

1 horas 4 minutos 55 segundos

Nota: Puede ayudar la división entera (//) y la función módulo (%)

```
In [74]: tiempo = Lambda x: print(x//3600, 'horas', x%3600//60, 'minutos', x%60, 'segundos')

tiempo(3895)

1 horas 4 minutos 55 segundos
```